C111 Assembly Language Programming
Fall 2026 Midterm Exam

Student Name: ID:

Total Points	

Note: This exam paper should be kept confidential and any dissemination violates copyright.

Q1	Q2	Q3	Q4	Q5	Q6
/10	/10	/20	/20	/20	/20

Q1 (10 points) Multiple-choice questions: enter your answer keys here:

1	2	3	4	5	6	7	8	9	10

For the following multiple-choice questions, each question has exactly one correct answer key. If multiple choices are correct, choose the option "All of the above". **Fill in the answer keys in the table above. (Answer keys written in the question area will not be counted.)**

1. In a 5-bit system, adding unsigned ints 29 and 5 sets which condition?
   A. No flags set
   B. Overflow flag set
   C. Carry flag set
   D. Zero flag set

2. In a 5-bit system, subtracting unsigned ints 4 - 7 results in which carry/borrow status?
   A. Carry=1 (Borrow=0)
   B. Carry=1 (Borrow=1)
   C. Carry=0 (Borrow=0)
   D. Carry=0 (Borrow=1)

3. Which statement is true when adding the signed ints 12 and 7 in a 5-bit system?
   A. Signed overflow occurs (V = 1)
   B. Carry out occurs (C = 1)
   C. The result is zero (Z = 1)
   D. The result 19 is represented correctly

4. In two's complement, TC(x) can be obtained by:
   A. Invert bits
   B. Invert bits and subtract one
   C. Invert bits and add one
   D. Add one then invert bits

5. What is the bit width of each register in ARM Cortex-M processors?
   A) 8 bits
   B) 32 bits
   C) 16 bits
   D) 64 bits

6. Which registers are considered "Low Registers" in ARM Cortex-M and can be accessed by any instruction?
   A) R0-R7
   B) R8-R12
   C) R13-R15
   D) R0-R12

7. What does ADC r1, r2, r3 compute?
   A. r1 = r2 − r3 + C
   B. r1 = r2 + r3
   C. r1 = r2 + r3 − C
   D. r1 = r2 + r3 + C

8. Which instruction performs reverse subtraction in ARM?
   A. SBC
   B. SUB
   C. RSB
   D. ADC

9. What happens when you execute LDRSH r1, [r0] and the halfword at the memory location has value 0x8123?
   A) r1 = 0x00008123
   B) r1 = 0xFFFF8123
   C) r1 = 0x81230000
   D) r1 = 0x00000123

10. In LDR r1, [r0, r2, LSL #3], what is the effective address used?
    A) r0 + r2
    B) r0 + (r2 × 2)
    C) r0 + (r2 × 4)
    D) r0 + (r2 × 8)

Q2. (10 points) Binary numbers

Assuming a 4-bit system. Show the equivalent decimal values when the data is interpreted as unsigned binary or signed binary.

Binary Value	Signed Decimal Value	Unsigned Decimal Value
1010		
0101		
1011		
0010		
0100		

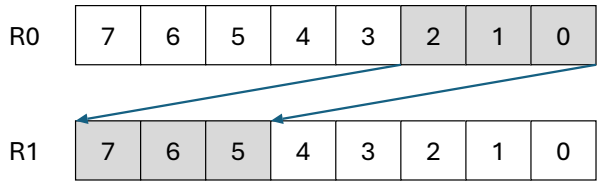
Q3. (20 points) Unsigned and signed arithmetic

Assume a 4-bit system. For each of the following operations, give the equivalent arithmetic calculation in decimal, and the NZCV flags, based on the first row that has been given to you. GT denotes Ground Truth value when there is a carry or overflow.

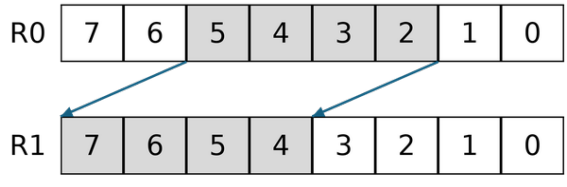
Operation	Result in binary	Equivalent unsigned arithmetic in decimal	Equivalent signed arithmetic in decimal	NZC V
0101 + 0011	1000	**5 + 3 = 8**	**5 + 3 = -8 (GT=8)**	**1001**
1000 - 0001	0111			
0110 - 1011	1011			
1010 + 1001	0011			
1101 + 0011	0000			
1011 - 0011	1000			

Q4. (20 points) Bit manipulations

(a) (5 points) Assume an 8-bit system (only the low 8 bits of R0 and R1 are meaningful). Copy the lowest three bits 2..0 in R0 to bits 7..5 in R1, and keep all other bits in R1 unchanged.



(b) (5 points) Assume an 8-bit system (only the low 8 bits of R0 and R1 are meaningful). Copy bits 5..2 in R0 to bits 7..4 in R1, and keep R1[3:0] unchanged.



(c) (10 points) Assume a 32-bit system. R0 and R1 are initialized with the values below. Write a sequence of assembly instructions that would produce the values in registers R2–R4 from R0 and R1. Do not use MOVW + MOVT. Do not use the pseudo-instruction LDR to load large constants into a register.

Given:

R0	0x1234ABCD
R1	0x0000FFFF

Produce:

R2	0x12340000
R3	0xABCD1234
R4	0xFFFF1234

Q5. (20 points) Program execution tracing

Assume a 32-bit system with **little-endian** byte order. The following assembly program operates on memory and registers. Fill in the values of registers R1 through R7 and the final memory contents after the entire program has finished executing. Write your answers in hexadecimal format.

Assembly program:

```
LDR   R1, =0x20000000
LDR   R2, [R1, #4]
LDRH  R3, [R1], #4
ASR   R4, R2, #8
MOV   R5, #0x0F
ORR   R6, R3, R5, LSL #8
EOR   R7, R2, R5
STR   R7, [R1, #4]
```

Initial memory contents:

Addr	Content															
0x20000000	11	22	33	44	9A	BC	DE	F0	55	66	77	88	99	AA	BB	CC

Final register values:

R1	R2	R3	R4
R5	R6	R7	

Final memory contents:

Addr	Content															
0x20000000																

Q6. (20 points) Assume a 32-bit system. Consider an array[] of 16 integers (each integer is 4 bytes). Assume register R0 holds the base memory address of array, i.e., array[0]. A compiler associates variables z and y with registers R1 and R2, respectively. Translate this C program into ARM assembly language based on the provided comments. Use R3 as a temporary register. (Note: Assume z is an unsigned integer. You may choose to use either pre-index, or pre-index with update, or post-index addressing.)

C program:

```
uint32_t z = y - array[1];
array[2] = z * 4;
array[3] = z / 8;
array[4] = z * 9;
array[5] = z & 0xFF;
```

Assembly program:

_____; load array[1] into temporary register R3

_____; z = y - array[1]

_____; R3 = z * 4

_____; store into array[2]

_____; R3 = z / 8

_____; store into array[3]

_____; R3 = z * 9

_____; store into array[4]

_____; R3 = z & 0xFF

_____; store into array[5]